

System Design Document

Trendify — E-commerce Web Platform

1. Purpose & Scope

This document describes how Trendify will be built, complementing the SRS (which describes what will be built). It covers system architecture, database design, API contracts, request/response flows, caching strategy, and deployment topology.

Scope covers both the current frontend-only phase (mock data) and the future backend phase (Node.js + PostgreSQL + Redis), so the frontend can be built against a stable contract from day one.

2. High-Level Architecture

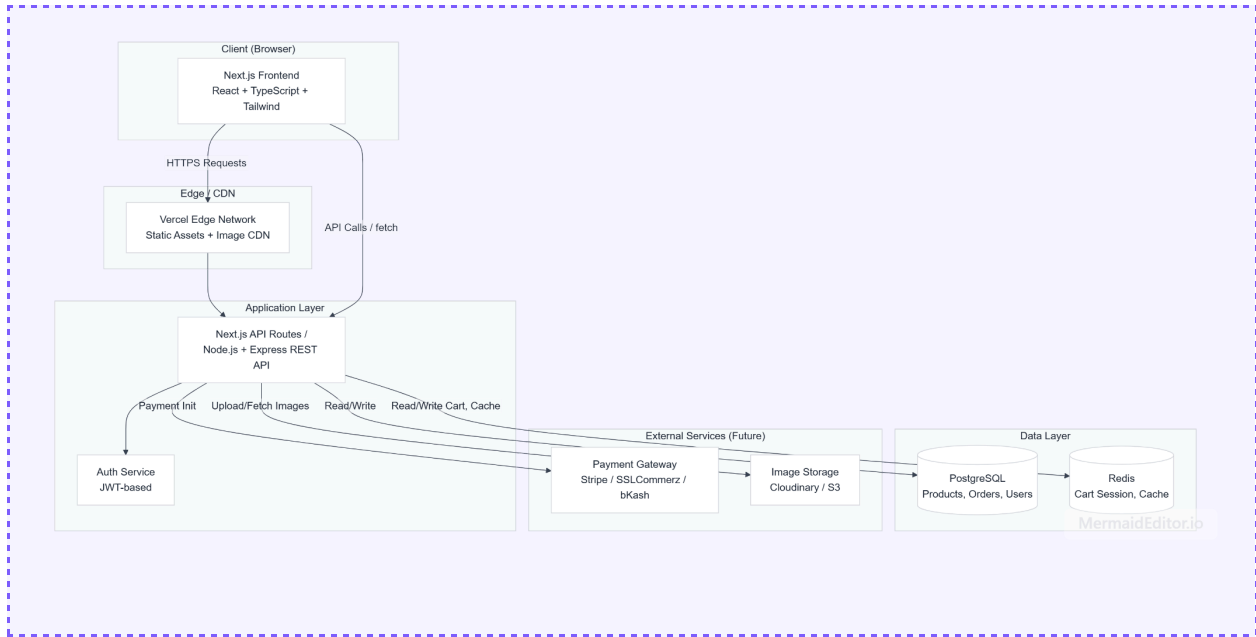
2.1 Architecture Style

Client-Server, 3-tier architecture with a caching layer:

- Presentation tier: Next.js (React, TypeScript, Tailwind) — SSR/SSG for product pages, CSR for interactive cart/wishlist
- Application tier: Node.js REST API (Express or Next.js API routes / Route Handlers)
- Data tier: PostgreSQL (persistent data) + Redis (cache/session/cart)

2.2 Architecture Diagram

 **DIAGRAM PLACEHOLDER**
High-Level System Architecture

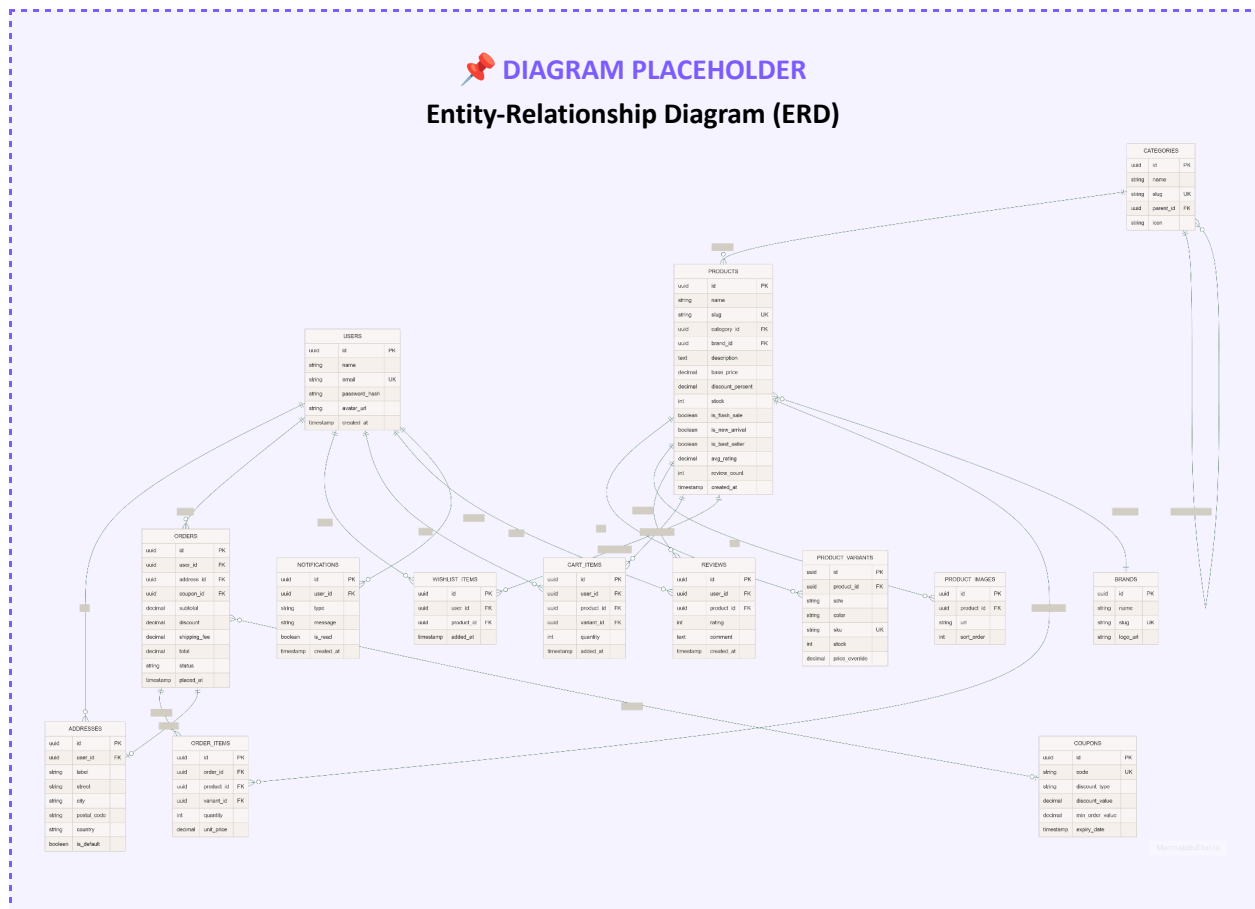


2.3 Component Responsibilities

Layer	Responsibility
Next.js Frontend	Rendering UI, client-side state (cart drawer, wishlist toggles, theme), calling API layer, SSR for SEO-critical pages (Home, PDP, Category)
API Layer	Business logic, validation, authentication, orchestrating DB/cache calls
PostgreSQL	Source of truth: users, products, orders, addresses, coupons, reviews
Redis	Fast-access cart sessions (esp. guest users), rate limiting, caching hot product lists (Best Deals, Flash Sale)
Payment Gateway	Handles payment processing (out of app's trust boundary — tokenized)
Image CDN	Serves optimized product images

3. Database Design (PostgreSQL)

3.1 Entity-Relationship Diagram



Core Entities Reference (for diagram accuracy check)

- **USERS** — id, name, email, password_hash, avatar_url, created_at
- **ADDRESSES** — id, user_id (FK), label, street, city, postal_code, country, is_default
- **CATEGORIES** — id, name, slug, parent_id (self FK), icon
- **BRANDS** — id, name, slug, logo_url
- **PRODUCTS** — id, name, slug, category_id (FK), brand_id (FK), description, base_price, discount_percent, stock, is_flash_sale, is_new_arrival, is_best_seller, avg_rating, review_count, created_at
- **PRODUCT_VARIANTS** — id, product_id (FK), size, color, sku, stock, price_override
- **PRODUCT_IMAGES** — id, product_id (FK), url, sort_order
- **CART_ITEMS** — id, user_id (FK), product_id (FK), variant_id (FK), quantity, added_at
- **WISHLIST_ITEMS** — id, user_id (FK), product_id (FK), added_at
- **ORDERS** — id, user_id (FK), address_id (FK), coupon_id (FK), subtotal, discount, shipping_fee, total, status, placed_at
- **ORDER_ITEMS** — id, order_id (FK), product_id (FK), variant_id (FK), quantity, unit_price
- **COUPONS** — id, code, discount_type, discount_value, min_order_value, expiry_date
- **REVIEWS** — id, user_id (FK), product_id (FK), rating, comment, created_at

- NOTIFICATIONS — id, user_id (FK), type, message, is_read, created_at

3.2 Key Design Notes

- PRODUCT_VARIANTS decouples size/color combinations from the base product so stock and SKU-level pricing can differ (e.g., one size might sell out while others remain).
- CART_ITEMS persists in Postgres for logged-in users but is mirrored/cached in Redis (keyed by session ID) for guest users, then merged into Postgres on login.
- price_override on variants allows optional per-variant pricing (e.g., larger sizes cost more); falls back to base_price if null.
- Soft-delete via a deleted_at timestamp is recommended for PRODUCTS and USERS rather than hard deletes (preserves order history integrity).
- Indexes recommended on: products.slug, products.category_id, products.is_flash_sale, orders.user_id, cart_items.user_id, coupons.code.

4. Caching Strategy (Redis)

Key Pattern	Purpose / TTL
cart:{sessionId}	Guest cart contents (JSON) — TTL 7 days
cart:user:{userId}	Logged-in user cart, write-through to Postgres — TTL 1 day
products:best-deals	Cached "Best Deals for You" list — TTL 5 minutes
products:recommended:{userId}	Cached personalized recommendations — TTL 15 minutes
flash-sale:active	Current active flash sale + end timestamp — TTL until sale ends
session:{token}	Auth session lookup — TTL matches JWT expiry
rate-limit:{ip}:{route}	Basic abuse/rate limiting on search & login — TTL 1 minute sliding window

Write pattern: Cart mutations write to Redis first (fast UX), then asynchronously sync to Postgres for logged-in users (write-behind), ensuring cart operations never feel slow even under DB load.

5. API Design (REST Contract)

Base URL: /api/v1

5.1 Products & Catalog

Method / Endpoint	Description
GET /products	List products (query: category, sort, page, limit, minPrice, maxPrice)
GET /products/:slug	Product detail
GET /products/best-deals	Curated deals section
GET /products/recommended	Personalized (auth) or trending (guest)
GET /products/flash-sale	Active flash sale items + countdown end time
GET /categories	List all categories
GET /search?q=	Search products by keyword

5.2 Cart

Method / Endpoint	Description
GET /cart	Get current cart (session or user-based)
POST /cart/items	Add item {productId, variantId, quantity}
PATCH /cart/items/:itemId	Update quantity
DELETE /cart/items/:itemId	Remove item
POST /cart/apply-coupon	Apply promo code {code}

5.3 Wishlist

Method / Endpoint	Description
GET /wishlist	Get saved items
POST /wishlist/:productId	Add to wishlist
DELETE /wishlist/:productId	Remove from wishlist

5.4 Orders & Checkout

Method / Endpoint	Description
POST /checkout	Create order from cart {addressId, paymentMethod}
GET /orders	List user's orders
GET /orders/:orderId	Order detail

5.5 User & Auth

Method / Endpoint	Description
POST /auth/register	Create account
POST /auth/login	Authenticate, returns JWT
POST /auth/logout	Invalidate session
GET /users/me	Current user profile
PATCH /users/me	Update profile
/addresses (GET/POST/PATCH/DELETE)	Address CRUD

5.6 Sample Contract — GET /products/:slug

```
{ "id": "uuid", "name": "Air Max 270 React", "slug": "air-max-270-react", "brand": "Nike",
"category": "Fashion", "price": 129.99, "originalPrice": 168.99, "discountPercent": 25,
"rating": 4.6, "reviewCount": 124, "images": ["https://cdn.../1.jpg",
"https://cdn.../2.jpg"], "variants": [ { "id": "uuid", "size": "7 US", "color": "White",
"stock": 12 } ], "description": "..."}

```

5.7 Sample Contract — POST /cart/items

Request:

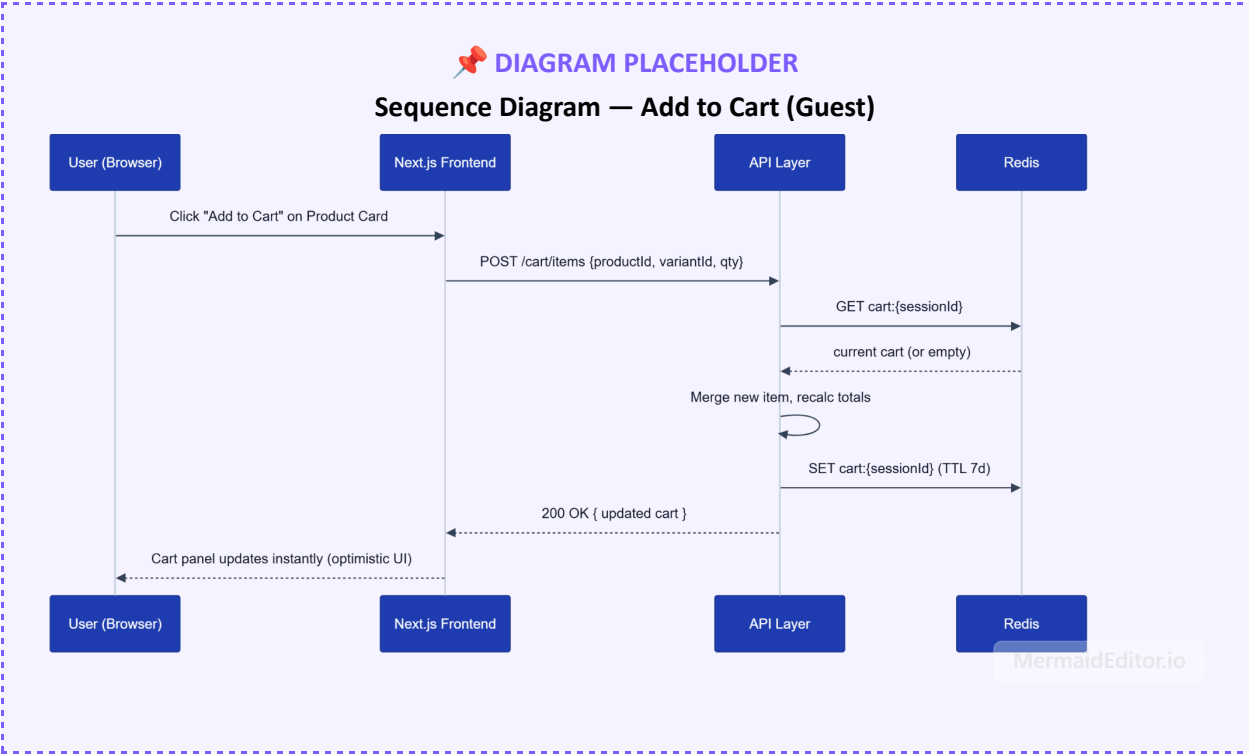
```
{ "productId": "uuid", "variantId": "uuid", "quantity": 1 }
```

Response:

```
{ "cart": { "items": [ { "id": "uuid", "name": "...", "unitPrice": 129.99, "quantity": 1 }
], "subtotal": 129.99, "discount": 0, "shipping": 0, "total": 129.99,
"itemCount": 1 }}
```

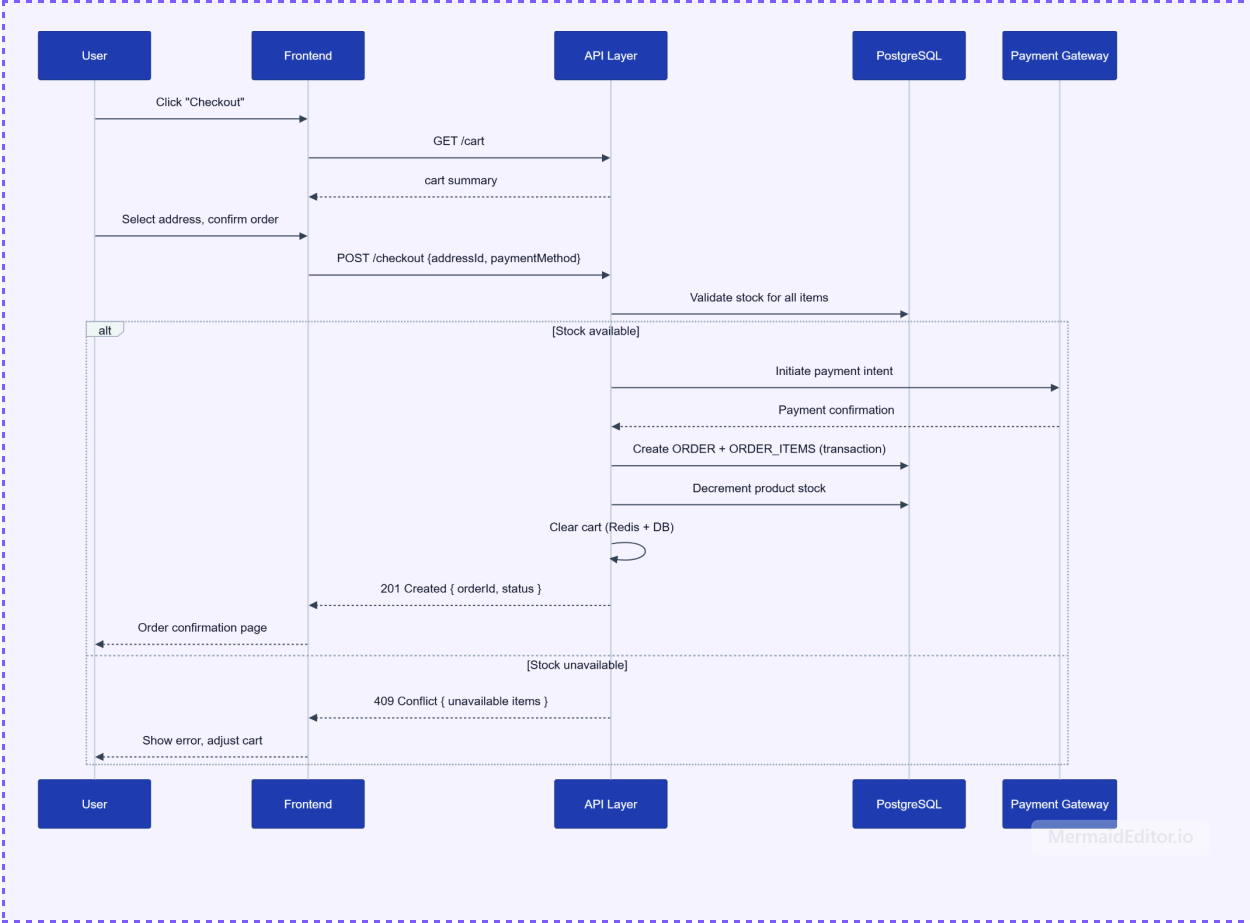
6. Key Sequence Diagrams

6.1 Add to Cart (Guest User)

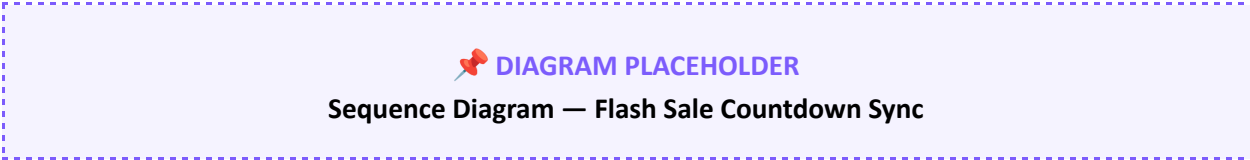


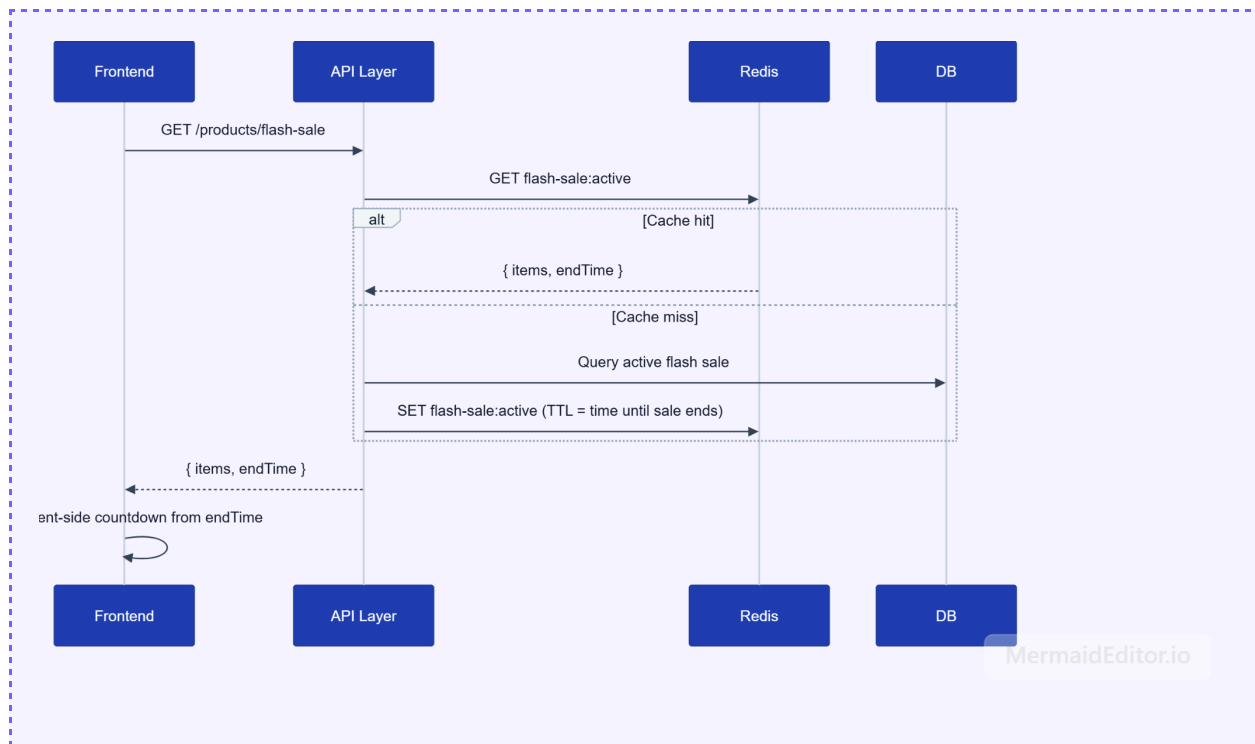
6.2 Checkout Flow (Logged-in User)



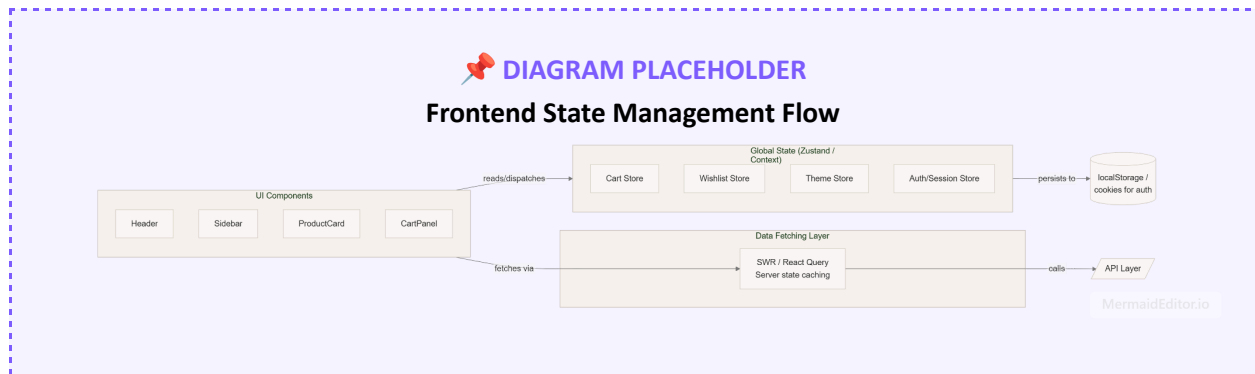


6.3 Flash Sale Countdown Sync





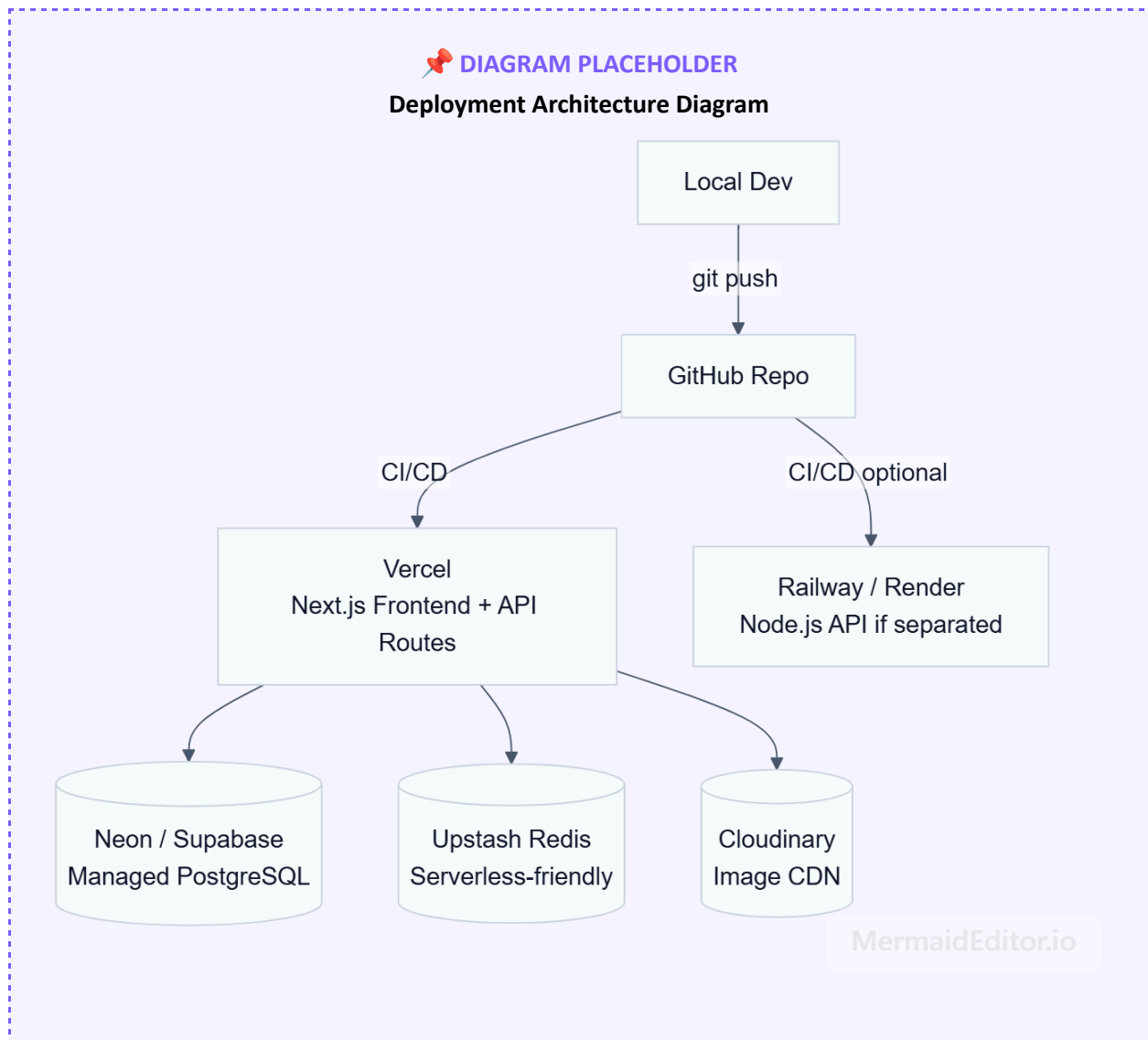
7. Frontend State Management Architecture



Rationale

- Zustand (lightweight) for client-only UI state: cart drawer open/close, theme, optimistic cart/wishlist counts.
- SWR or React Query for server state: product lists, order history — built-in caching, revalidation, loading/error states.
- Auth state persisted via httpOnly cookie (JWT) once backend exists; never stored in plain localStorage.

8. Deployment Architecture (Recommended)



Notes for Your Stack

- Given your prior use of Upstash/Redis, this fits directly — Upstash is serverless-friendly and pairs well with Vercel's edge functions (no persistent connection issues).
- Neon or Supabase for managed Postgres avoids needing to self-host a DB during the portfolio phase.
- Keep API routes inside the Next.js app (`/app/api/...`) initially for simplicity; extract to a standalone Express service only if the API grows complex enough to warrant independent scaling/deployment.

9. Non-Functional Considerations (System-Level)

Concern	Approach
Scalability	Stateless API layer (horizontally scalable); Redis absorbs cart/read-heavy load; Postgres read replicas if traffic grows (future)
Consistency	Cart writes are write-behind (Redis first, Postgres eventually) — acceptable since cart is not financially final until checkout, which is a DB transaction
Security	JWT auth with short-lived access tokens + refresh tokens; parameterized queries (ORM: Prisma recommended); rate limiting on /auth and /search
Observability	Structured logging (API layer), error tracking (Sentry), basic uptime monitoring
Data integrity	Order creation wrapped in a DB transaction (stock check + order insert + stock decrement atomic)

10. Suggested Tech Stack Summary

Layer	Technology
Frontend	Next.js (App Router), TypeScript, Tailwind CSS
State (client)	Zustand
State (server/cache)	SWR or React Query
Backend API	Next.js Route Handlers (or Express if separated)
ORM	Prisma (maps cleanly to the ER diagram above)
Database	PostgreSQL (Neon / Supabase)
Cache/Session	Redis (Upstash)
Auth	JWT (NextAuth.js optional for social login later)
Image hosting	Cloudinary or Vercel Image Optimization
Deployment	Vercel
Testing (future)	Playwright + Page Object Model

11. Document Control

Version	Date / Change
1.0	2026-08-16 — Initial System Design created, companion to Trendify SRS v1.0